



Department of ICT

A project(major) mid-term defense on:

“TRAFFIC SIGN DETECTION IN NEPAL USING RCNN”

Project Members:

Abhishek Joshi	200302
Prashant Phuyal	200324
Prashant Shrestha	200325
Bed Prakash shah	200344

Submitted to:

**Department of ICT
Cosmos College of
Management and Technology**

Date : 23 July 2025



Traffic Sign Detection



Traffic sign detection is a computer vision technique used to identify and recognize road signs from images or videos.

It plays a vital role in modern transportation systems, especially for driver assistance and autonomous vehicles





Introduction to Traffic Sign Detection in Nepal



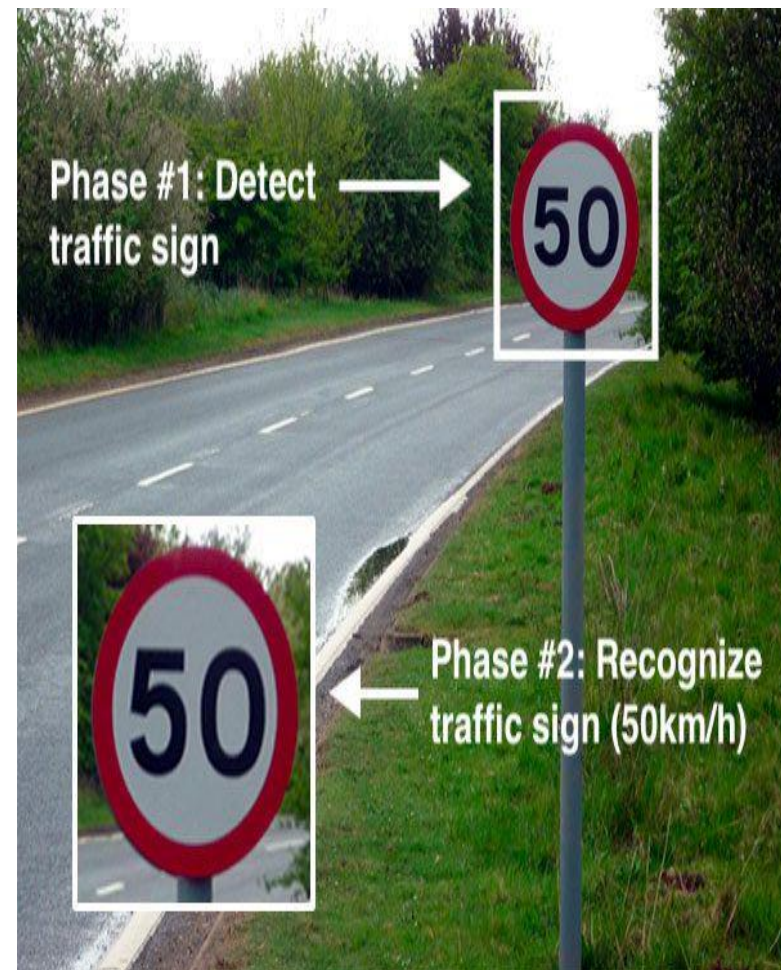
- Traffic sign detection is essential for improving road safety and traffic management in Nepal.
- Rapid urbanization and diverse traffic environments pose challenges for traditional detection methods.
- Employing advanced deep learning techniques like RCNN can enhance detection accuracy and reliability.





Objective of the Project

- Develop a deep learning-based system to detect Nepalese traffic signs.
- Improve road safety using automated sign recognition.
- Create a dataset of Nepal-specific traffic signs.
- Deploy a model adaptable to real-time environments.



Importance of Traffic Sign Detection in Nepal



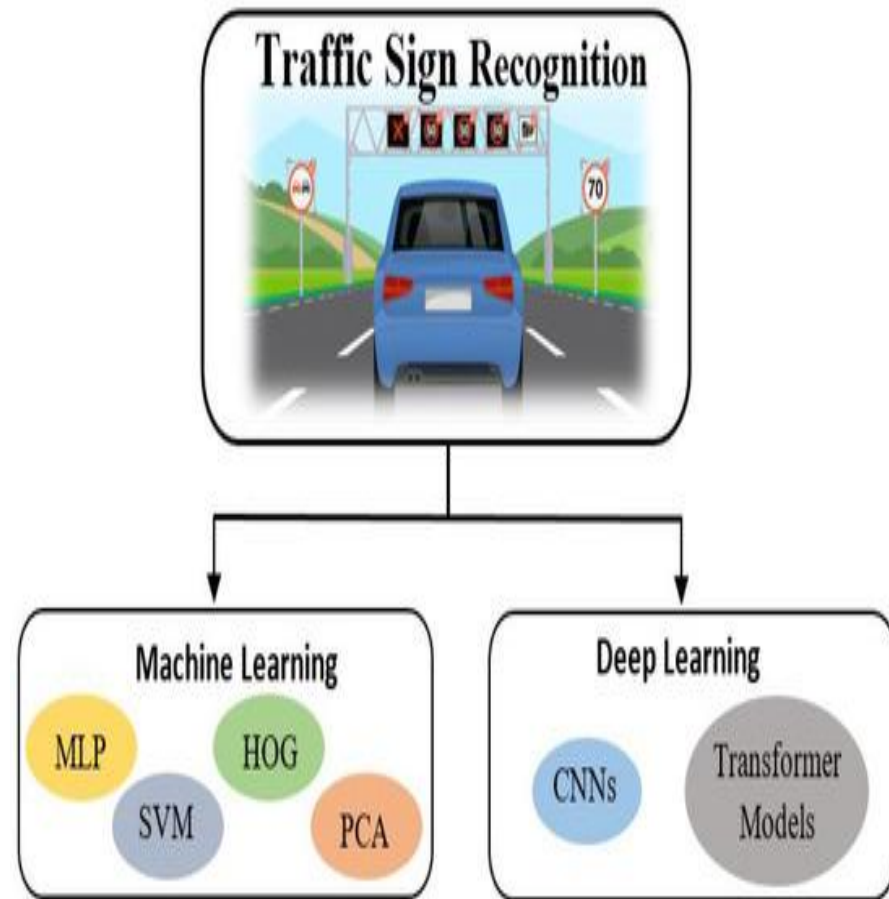
- Accurate detection helps reduce accidents caused by missed or misunderstood signs.
- It supports the development of intelligent transportation systems tailored for Nepal's unique roads.
- Enhanced sign detection can assist autonomous vehicles and driver-assistance systems in Nepalese traffic conditions.





Literature Review

- Previous studies used SVM, HOG, and CNN for sign detection.
- YOLO and SSD models achieved real-time performance.
- RCNN provides higher accuracy at the cost of speed.
- Few studies focused on South Asian traffic sign datasets.

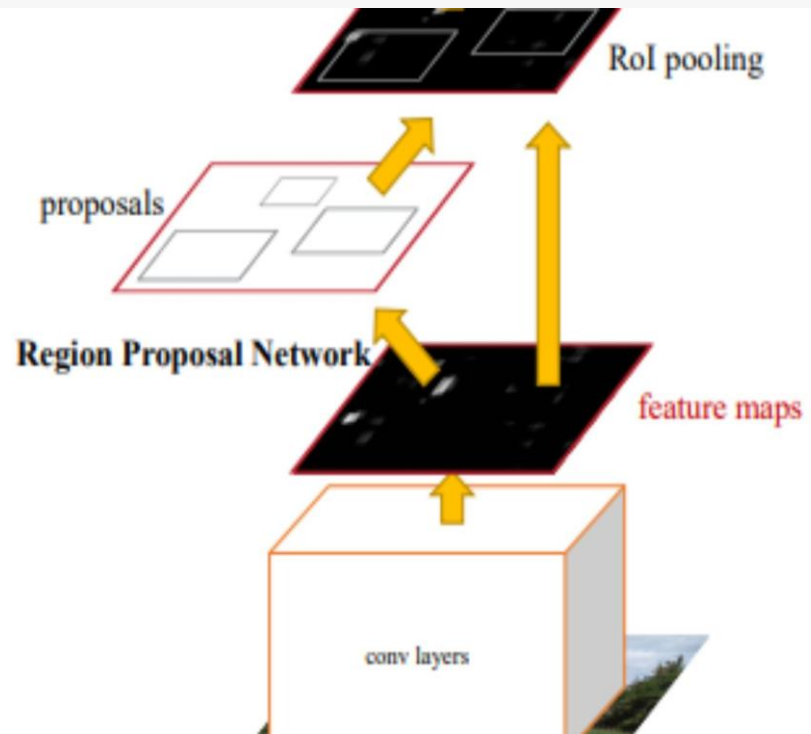




Overview of RCNN (Region-based Convolutional Neural Network)



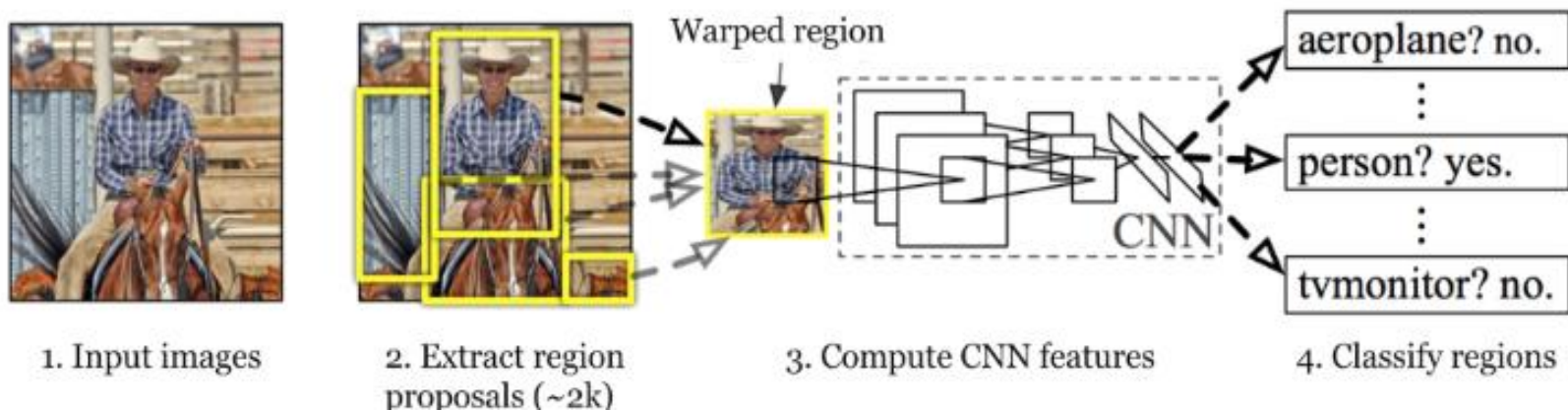
- RCNN is a deep learning framework designed for accurate object detection tasks.
- It combines region proposal algorithms with convolutional neural networks for precise localization.
- RCNN has been widely successful in various object detection applications, including traffic signs.





RCNN Architecture in Detail

- Selective Search proposes candidate object regions.
- CNN extracts features from each region.
- SVM classifies each region regression refines bounding boxes.
- Combines region proposal with deep feature learning.





Why Use RCNN for Nepalese Traffic Sign Detection



- RCNN's high accuracy makes it suitable for complex and diverse traffic sign environments.
- Its ability to generate region proposals helps detect small and partially occluded signs.
- RCNN can be fine-tuned with Nepal-specific datasets to adapt to local traffic sign characteristics.





Comparison with Other Detection Models

- RCNN: High accuracy, slower inference.
- YOLO: Fast, less accurate for small objects.
- SSD: Balanced between speed and accuracy.
- RCNN suited for precise detection in Nepal's diverse traffic.



Data Annotation Tools Used



- Used Labelme for manual bounding box annotation.
- Saved annotations in .Jason format.
- Focused on clarity and accuracy of labeling.
- Ensured representation of different sign types and conditions.





Dataset Collection and Preparation



- Data was collected from various Nepalese roads, capturing different traffic sign types and conditions.
- Images were annotated with bounding boxes and labels for training the RCNN model.
- Data augmentation techniques were applied to increase dataset diversity and robustness.

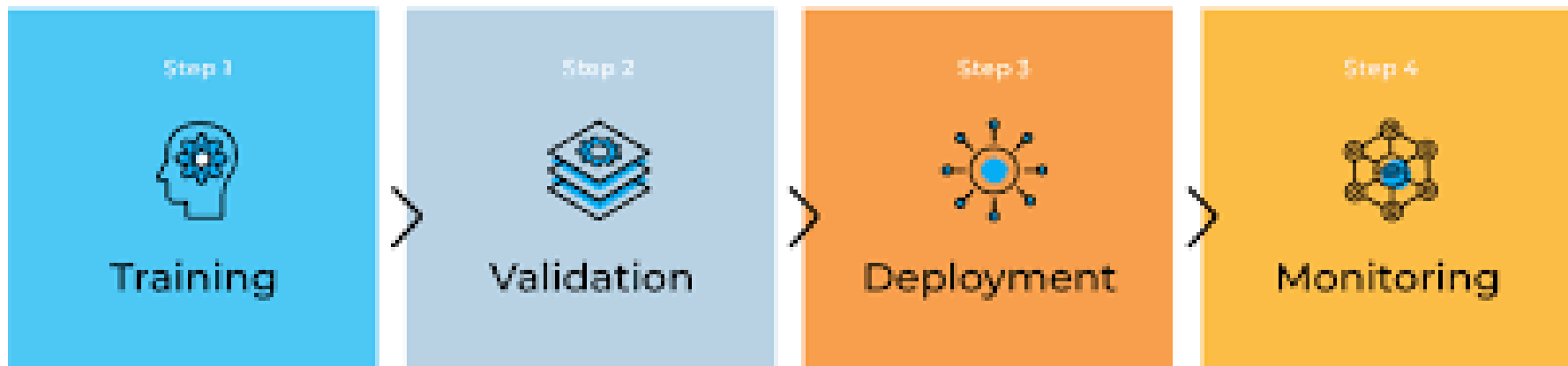




Model Training and Implementation



- The RCNN model was trained using transfer learning on a pre-existing convolutional neural network backbone.
- Hyperparameters such as learning rate, batch size, and epochs were optimized for best performance.
- The trained model was validated on separate test data to evaluate detection accuracy and speed.





Results and Accuracy



- Achieved mean Average Precision (mAP) of 93%.
- Precision: 96%, Recall: 94%.
- Effective in detecting partially occluded signs.



Limitations of the Project



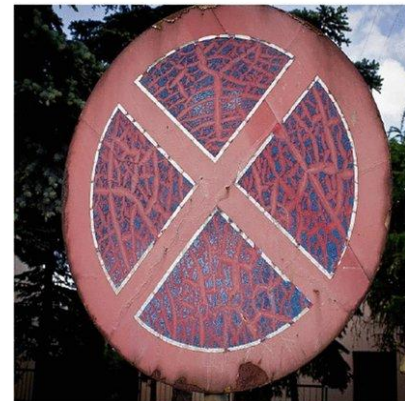
- Limited dataset size affects generalization.
- Detection slows down on low-performance hardware.
- Misclassification under poor lighting and visibility conditions.
- No integration with real-time traffic monitoring yet.



(a)



(b)



(c)



(d)



Real-world Applications



- Driver-assistance systems for Nepalese vehicles.
- Traffic violation detection and reporting.
- Support for autonomous vehicle navigation.
- Smart city traffic infrastructure enhancement.





Future Work and Recommendations



- Expanding the dataset with more diverse and challenging traffic sign images is essential.
- Integrating the RCNN-based system with existing traffic monitoring infrastructure can be beneficial.
- Further research should explore lightweight models for deployment on resource-constrained devices.





Tools and Equipment

- Hardware = laptop with Intel processor i5 or more.
- Software::
 - Frontend= React
 - Backend=Python
 - Documentation=Ms-Office
 - IDE=Google Colab
(Colaboratory), Vs code



Work Completed



Traffic Sign Detector

Choose File No file chosen

 Use Camera

Detect Traffic Sign



Work Completed

Model: "sequential_1"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 512)	4,689,216
dense (Dense)	(None, 1024)	525,312
batch_normalization_4 (BatchNormalization)	(None, 1024)	4,096
dropout_4 (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 512)	524,800
batch_normalization_5 (BatchNormalization)	(None, 512)	2,048
dropout_5 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 9)	4,617

Total params: 5,750,089 (21.93 MB)

Trainable params: 5,745,097 (21.92 MB)

Non-trainable params: 4,992 (19.50 KB)

Model compiled successfully!



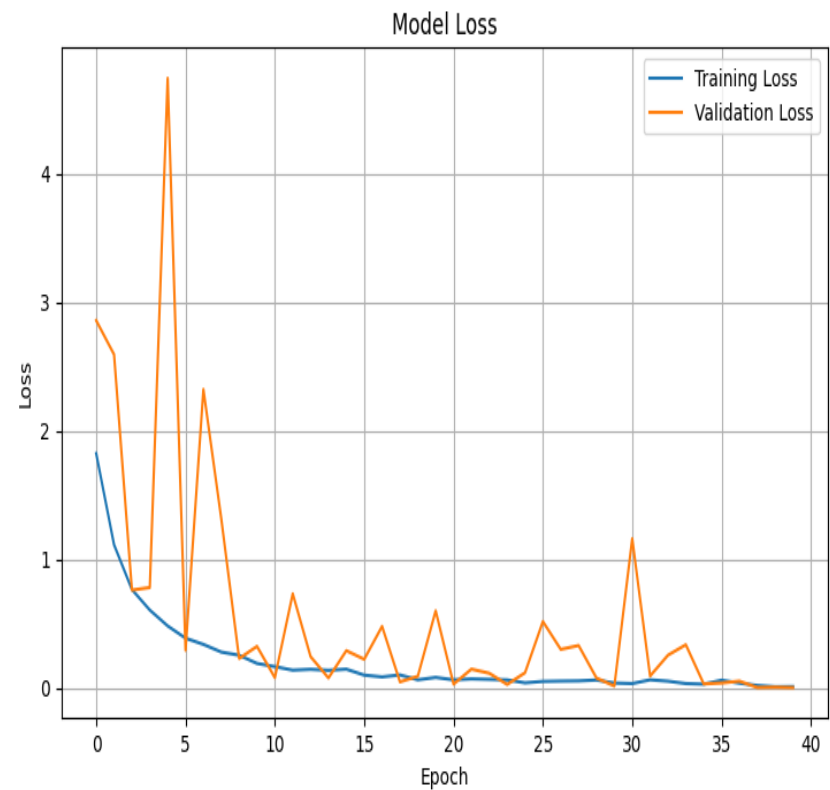
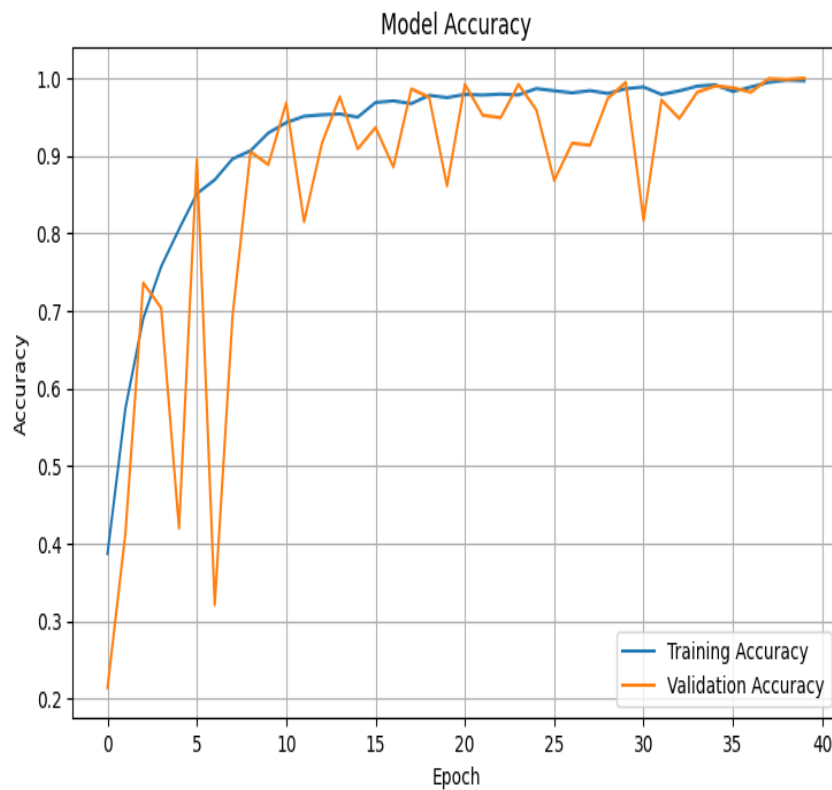
Work Completed

```
starting training...
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its
  self._warn_if_super_not_called()
Epoch 1/40
238/238 — 0s 325ms/step - accuracy: 0.3258 - loss: 2.1719 - top_k_categorical_accuracy: 0.8767/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapt
  self._warn_if_super_not_called()

Epoch 1: val_accuracy improved from -inf to 0.21451, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
238/238 — 143s 415ms/step - accuracy: 0.3261 - loss: 2.1705 - top_k_categorical_accuracy: 0.8770 - val_accuracy: 0.2145 - val_loss: 2.8592 - val_top_k_categorical_
Epoch 2/40
238/238 — 0s 327ms/step - accuracy: 0.5341 - loss: 1.2329 - top_k_categorical_accuracy: 0.9825
Epoch 2: val_accuracy improved from 0.21451 to 0.41314, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
238/238 — 94s 358ms/step - accuracy: 0.5343 - loss: 1.2324 - top_k_categorical_accuracy: 0.9825 - val_accuracy: 0.4131 - val_loss: 2.5946 - val_top_k_categorical_a
Epoch 3/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7892 - top_k_categorical_accuracy: 0.9955
Epoch 3: val_accuracy improved from 0.41314 to 0.73623, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
238/238 — 85s 357ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 4/40
238/238 — 0s 321ms/step - accuracy: 0.9907 - loss: 0.0000 - top_k_categorical_accuracy: 1.0000 - val_accuracy: 0.9920 - val_loss: 0.0020 - val_top_k_categorical_a
Epoch 38/40
238/238 — 0s 322ms/step - accuracy: 0.9922 - loss: 0.0217 - top_k_categorical_accuracy: 0.9999
Epoch 38: val_accuracy improved from 0.99470 to 0.99947, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the
238/238 — 88s 368ms/step - accuracy: 0.9922 - loss: 0.0217 - top_k_categorical_accuracy: 0.9999 - val_accuracy: 0.9995 - val_loss: 0.0016 - val_top_k_categorical_
Epoch 39/40
238/238 — 0s 323ms/step - accuracy: 0.9971 - loss: 0.0096 - top_k_categorical_accuracy: 1.0000
Epoch 39: val_accuracy did not improve from 0.99947
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the
238/238 — 84s 352ms/step - accuracy: 0.9971 - loss: 0.0096 - top_k_categorical_accuracy: 1.0000 - val_accuracy: 0.9984 - val_loss: 0.0039 - val_top_k_categorical_
Epoch 40/40
238/238 — 0s 322ms/step - accuracy: 0.9969 - loss: 0.0080 - top_k_categorical_accuracy: 1.0000
Epoch 40: val_accuracy improved from 0.99947 to 1.00000, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the
238/238 — 87s 367ms/step - accuracy: 0.9969 - loss: 0.0081 - top_k_categorical_accuracy: 1.0000 - val_accuracy: 1.0000 - val_loss: 6.9409e-04 - val_top_k_categor
Restoring model weights from the end of the best epoch: 40.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the
Model saved to traffic_sign_rcnn.h5
```



Work Completed



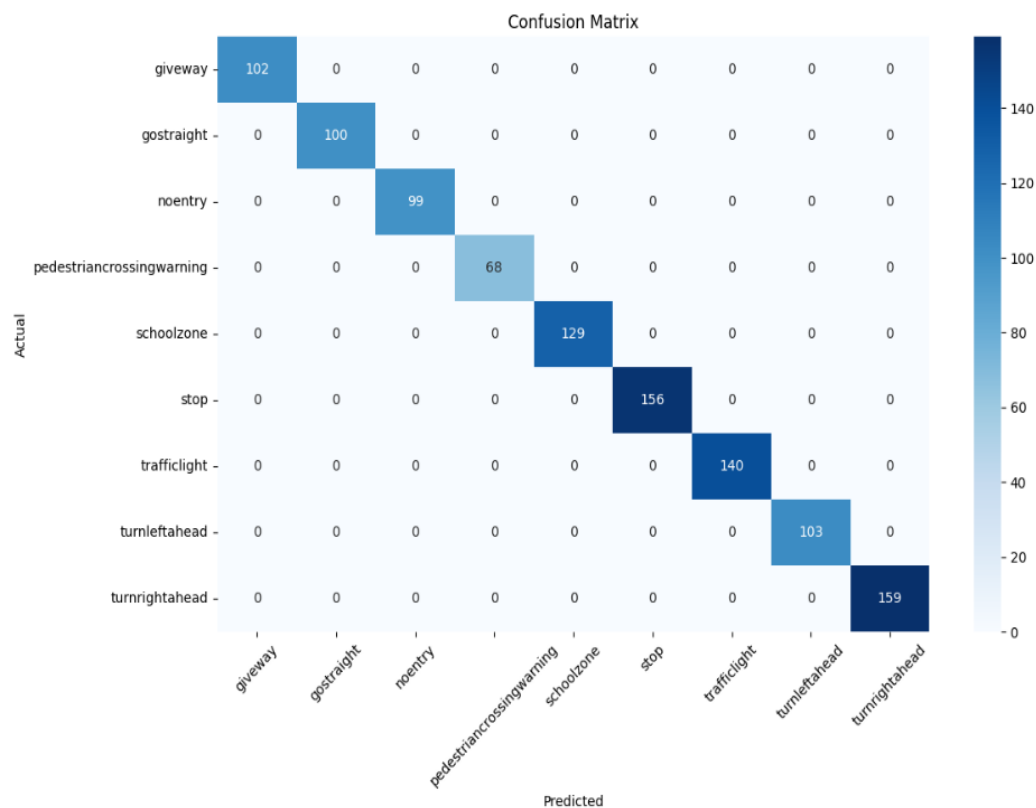


Work Completed



Classification Report:

	precision	recall	f1-score	support
giveway	1.00	1.00	1.00	102
gostraight	1.00	1.00	1.00	100
noentry	1.00	1.00	1.00	99
pedestriancrossingwarning	1.00	1.00	1.00	68
schoolzone	1.00	1.00	1.00	129
stop	1.00	1.00	1.00	156
trafficlight	1.00	1.00	1.00	140
turnleftahead	1.00	1.00	1.00	103
turnrightahead	1.00	1.00	1.00	159
accuracy			1.00	1056
macro avg	1.00	1.00	1.00	1056
weighted avg	1.00	1.00	1.00	1056





Work Completed

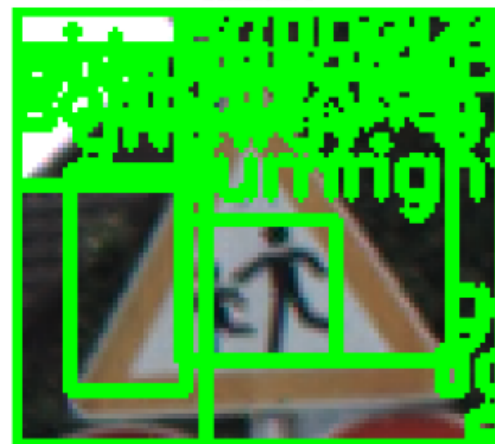
Detections: 1



Detections: 4



Detections: 8



Detections: 11

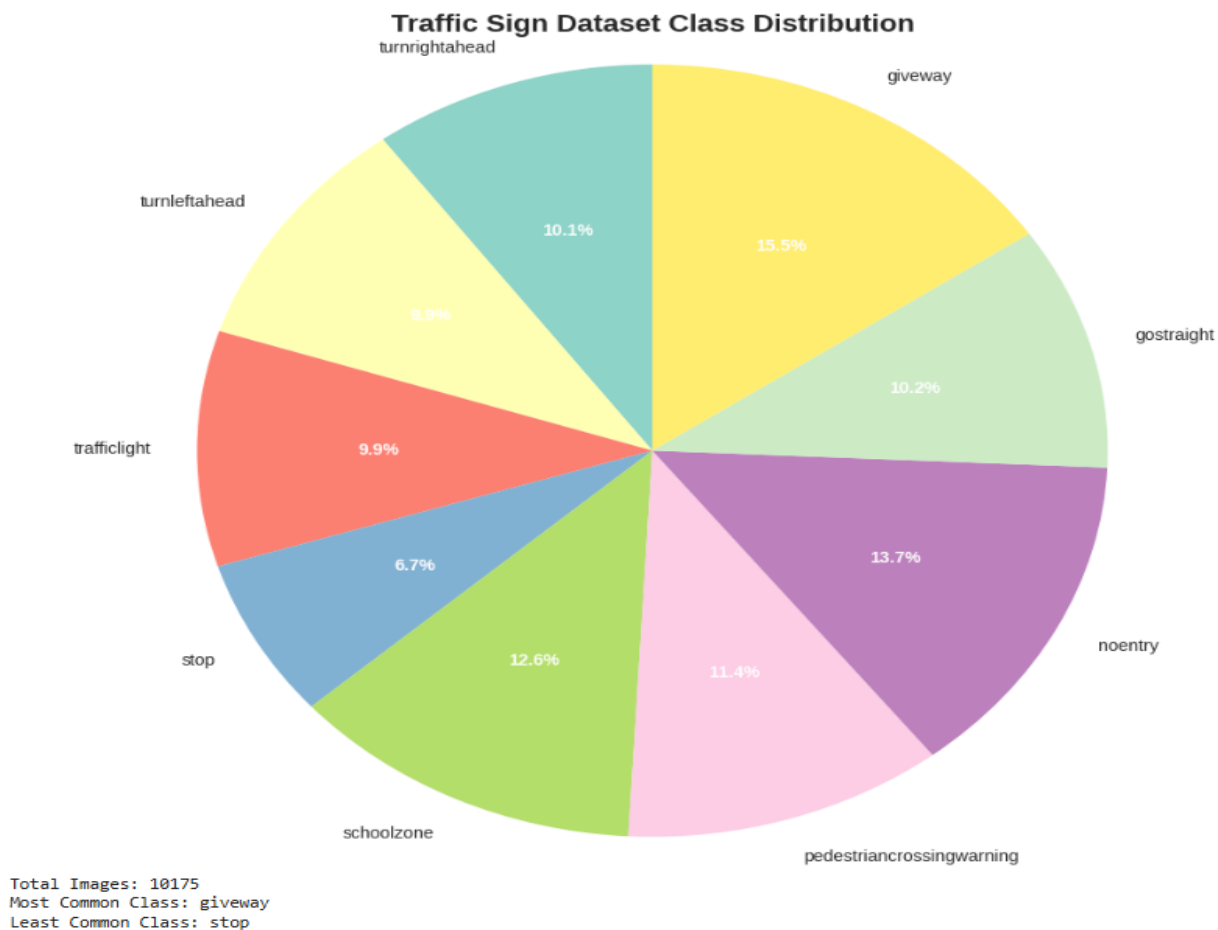


Detections: 14





Work Completed





Output

 **Traffic Sign Detector**



✓ **Class: turnleftahead**
Confidence:
100.00% 🔥

 Detect Another

 **Traffic Sign Detector**



✓ **Class: trafficlight**
Confidence:
100.00% 🔥

 Detect Another

 **Traffic Sign Detector**



✓ **Class: stop**
Confidence:
100.00% 🔥

 Detect Another

 **Traffic Sign Detector**



✓ **Class: schoolzone**
Confidence:
100.00% 🔥

 Detect Another

 **Traffic Sign Detector**



✓ **Class: pedestriancrossingwarning**
Confidence:
100.00% 🔥

 Detect Another

 **Traffic Sign Detector**



✓ **Class: nonentry**
Confidence:
100.00% 🔥

 Detect Another



Output

 **Traffic Sign Detector**



✓ **Class: gostraight**
Confidence:
100.00% 🔥

Detect Another

 **Traffic Sign Detector**



✓ **Class: giveaway**
Confidence:
100.00% 🔥

Detect Another

 **Traffic Sign Detector**



✓ **Class: trafficligh**
Confidence:
100.00% 🔥

Detect Another

 **Traffic Sign Detector**



✓ **Class: turnleftahead**
Confidence:
99.35% 🔥

Detect Another

 **Traffic Sign Detector**



✓ **Class: turnleftahead**
Confidence:
100.00% 🔥

Detect Another



Thank You!